



12. Adaptive Boosting

What if we focus on the learner itself, rather than the mistake it makes?

Previous Section:

 [11. Gradient Boosting](#)

Contents:

[1. Adaptive Boosting vs Gradient Boosting](#)

[2. Adaptive Boosting: The Workflow](#)

[3. Adaptive Boosting with Python](#)

[Summary](#)

[References](#)

In this section, we talk another really well-known boosting method: Adaptive Boosting, or what people usually call AdaBoost.

Now, if you remember the previous section, we spent quite a bit of time talking about Boosting, and especially Gradient Boosting. The main idea there was very clear:

We build models **sequentially**, and each new model tries to fix the mistakes of the previous ones.

In other words, we focused heavily on the **errors**. But here's where things get interesting. What if instead of focusing on the mistakes directly, we shift our attention to something else?

What if we focus on the **learner itself**?

Not “what did you get wrong?” → But “how should you pay attention next time?”

That subtle shift is exactly what Adaptive Boosting is about.

Adaptive Boosting is still part of the Boosting family—so it follows the same high-level idea of combining multiple weak learners to form a strong one. But instead of correcting errors using gradients (like Gradient Boosting does), AdaBoost does something more strategic. It ***reweights the data***.

- Points that are correctly classified? → They become less important
- Points that are misclassified? → They become more important

So the next learner doesn’t just “try again”. It is **forced to focus** on the hard cases.

And this is why the name “Adaptive” makes so much sense. Because the model is constantly adapting— not just improving, but **shifting its attention** based on what matters most.

So instead of saying: “Let’s fix the error mathematically” (Gradient Boosting)

AdaBoost says: “Let’s change what the learner sees as important.”

Over time, this creates a very powerful effect. Each weak learner becomes slightly better, because it is trained on a version of the dataset that highlights the difficult parts.

And when you combine all of them together? You don’t just get a model that performs well. You get a model that has **learned where to look**.

1. Adaptive Boosting vs Gradient Boosting

Adaptive Boosting, or AdaBoost, is one of those methods that sounds simple on the surface—but once you really look at it, you realize it’s doing something very clever behind the scenes.

It is an ensemble learning method, mainly designed for classification problems. And yes, just like Gradient Boosting, it builds multiple models and combines them together.

But again, don’t rush past that similarity. Because the *way* it learns is very different. AdaBoost is an **iterative ensemble technique**. It takes a set of weak learners—models that, by themselves, perform only slightly better than random guessing—and turns them into something much stronger.

And the key idea is this: It doesn't just learn from mistakes. It **changes what matters** after each iteration.

How AdaBoost Actually Works - Let's walk through it slowly, the way Iris would observe it.

At the beginning, everything is fair.

- Every data point is treated equally
- Every sample has the same weight

No bias. No preference. Just a clean start.

- **Step 1 — First Learner:** We train the first weak learner using the full dataset. At this point, it doesn't know anything. So naturally... it will make mistakes.
- **Step 2 — Observe the Mistakes:** Now we do something very important. We don't just say "okay, there are errors". We ask:
 - Which samples were classified correctly?
 - Which ones were misclassified?

Because not all data points are equal anymore.

- **Step 3 — Shift the Attention:** Here's where AdaBoost becomes *adaptive*.
 - Misclassified samples → their weights increase
 - Correctly classified samples → their weights decrease

So what happens? The dataset itself changes—not in values, but in **importance**. The next learner will see the same data, but it will **care more** about the difficult cases.

- **Step 4 — Evaluate the Learner:** Now we evaluate how good that weak learner was.
 - If it performs well → it gets a higher weight
 - If it performs poorly → it gets a lower weight

So now, not only do data points have weights—**learners also have weights**.

- **Step 5 — Repeat the Process:** We create a new weighted dataset, and train the next weak learner. And again:
 - Focus more on the hard samples → Adjust learner importance → Repeat. Over multiple iterations, the model becomes sharper, not by being perfect, but by being **focused**.
- **Step 6 — Final Combination:** At the end, we combine all weak learners. But not equally. Each learner “votes”, and its vote is scaled by how good it was.
 - So the final prediction is not just a simple average—it’s a **weighted decision** built from experience.

Important Distinction: Two Types of Weights

This is something students often confuse, so I will pause here. There are **two different types of weights** in AdaBoost:

- **Sample Weights:** Belong to the data → Change every iteration → Control *what the next learner focuses on*
- **Learner Weights:** Belong to the models → Determined by accuracy → Control *how much each learner influences the final prediction*

One controls attention. The other controls influence.

Think of AdaBoost like a teacher. At the start, every student is treated equally.

After the first test: Some students did well and some students struggled.

So what does the teacher do? They don’t reteach everything equally. They focus more on the students who got things wrong.

Next round: Struggling students → more attention; Strong students → less focus

And each teaching session (each weak learner):

- If it works well → it matters more
- If it doesn’t → it matters less

After many rounds, the teacher doesn't just "teach better"... They understand **who needs attention**. And that's what makes the final outcome strong.

Now let's connect this back to what we learned before to compare: **AdaBoost vs Gradient Boosting**.

Both methods improve sequentially, but their philosophy is completely different.

Aspect	AdaBoost	Gradient Boosting
How the model improves	Adjusts sample weights so future learners focus on misclassified data	Learns residuals (errors) directly
Model used	Usually Decision Stumps (very simple trees)	Typically deeper decision trees
Optimization	Assigns weights to learners based on accuracy	Minimizes a loss function using gradients
Where attention goes	Focus on <i>data points</i> (hard samples)	Focus on <i>errors</i> (residuals)

AdaBoost asks: → *"Which samples should I care about more?"*

Gradient Boosting asks: → *"What mistakes should I fix next?"*

And that difference changes everything. Because one adjusts **attention** while the other adjusts **correction**.

AdaBoost is not just about combining weak learners. It's about **learning where to look**. Because sometimes, the problem is not that the model is weak. It's that the model is looking at everything instead of focusing on what truly matters.

2. Adaptive Boosting: The Workflow

Let's stop talking in theory for a moment. Because AdaBoost only truly makes sense when you **watch it happen**. So I will do what I do best: Observe. Track. Reconstruct the process step by step.

- **Step 1 — Assign Initial Sample Weights:** At the beginning, everything is neutral. We don't assume anything. We don't favor any data point.

So every sample is given the same importance. We have $N = 10$ **samples**, so each one receives:

$$w = 1/N = 1/10 = 0.1$$

```

import pandas as pd

data = {"Person": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
        "Red": [1, 1, 1, 1, 0, 1, 1, 0, 0, 1],
        "Yellow": [1, 0, 1, 1, 1, 0, 1, 1, 0, 1],
        "Green": [1, 1, 0, 1, 1, 0, 0, 1, 1, 1],
        "Blue": [1, 1, 1, 0, 1, 1, 0, 0, 1, 1],
        "Target": ["Not Colorblind", "Not Colorblind", "Not Colorblind",
                    "Not Colorblind", "Not Colorblind", "Colorblind", "Colorblind", "Colorblind", "Not Colorblind", "Not Colorblind"]}

df = pd.DataFrame(data)

# We use Label Encoding for demonstration only
for i in df.columns:
    unique_columns = df[i].unique()
    df[i] = df[i].map({unique_columns[value]: value for value in range(len(unique_columns))})

df['Weight'] = 1 / len(df)
print(df)

```

	Person	Red	Yellow	Green	Blue	Target	Initial Weight
0	0	0	0	0	0	0	0.1
1	1	0	1	0	0	0	0.1
2	2	0	0	1	0	0	0.1
3	3	0	0	0	1	0	0.1
4	4	1	0	0	0	0	0.1
5	5	0	1	1	0	1	0.1
6	6	0	0	1	1	1	0.1
7	7	1	0	0	1	1	0.1
8	8	1	1	0	0	1	0.1
9	9	0	0	0	0	0	0.1

This might look trivial, but it's not. This is the **baseline of fairness**. And one rule must always hold → The total weight must equal 1. Not approximately. Not

"close enough." → Exactly 1.

Because AdaBoost doesn't just train on data. It trains on **probability distribution over data**.

- **Step 2 — Train the First Weak Learner:** Now we train the first weak learner.

Nothing fancy. Just a simple model.

True Label	Predicted Label	Evaluation
0	0	
0	1	Incorrect
0	0	
0	0	
0	0	
1	0	Incorrect
1	1	
1	1	
1	1	
0	1	Incorrect

And as expected, it makes mistakes. Out of 10 samples → it misclassifies **3** → We then calculate the **weighted error**:

$$\epsilon = 0.1 \times 3 = 0.3$$

Now this is important.

- If $\epsilon \geq 0.5$ → the model is useless (worse than guessing)
- If $\epsilon < 0.5$ → the model is usable

Here: $0.3 < 0.5$ → acceptable.

Now comes the key question: "How much should we trust this learner?"

AdaBoost answers that with a weight:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon}{\epsilon} \right)$$

Substitute $\epsilon = 0.3$ → $\alpha_t \approx 0.42365$

What does this mean? Not just “a number”. It means:

- This learner is **better than random**
- It deserves a **moderate influence**

Not dominant. Not negligible. But **trusted**.

- **Step 3 — Update Sample Weights**

Now we do something very specific. We do not change the model. We change the **importance of the data**.

(A) Misclassified Samples: These are the “problem cases.” They were misunderstood. So AdaBoost increases their importance:

$$w_{new} = 0.1 \times e^{(\alpha_t)} \approx 0.15275$$

These points now carry more weight. Not because they are many. But because they are **difficult**.

(B) Correctly Classified Samples: These are already handled well.

So their importance is reduced:

$$w_{new} = 0.1 \times e^{(-\alpha_t)} \approx 0.065465$$

They don’t disappear. They just become less urgent.

(C) Check Total Weight: Now we check something critical. The total weight:

$$W = 3(0.15275) + 7(0.065465) = 0.916505$$

And immediately, we see the issue → This is not 1. So the distribution is invalid.

- **Step 4 — Normalize the Weights**

AdaBoost fixes this by scaling everything back.

- Misclassified samples: $w = 0.15275/0.916505 \approx 0.1667$
- Correct samples: $w = 0.065465/0.916505 \approx 0.0714$

Now check again: $3 \times 0.1667 + 7 \times 0.0714 \approx 1 \rightarrow$ Balance restored.

The Real Effect (This is the Important Part)

- Before: Every sample = 0.1 → Equal importance
- After: Misclassified = 0.1667; Correctly Misclassified = 0.0714

→ Let that sink in. The difficult samples now have **more than double the importance**. And this is where AdaBoost becomes dangerous (in a good way). Because the next learner is no longer trained on “data”. It is trained on **what the previous learner failed to understand**.

We can update our data as follows:

```
df['Predicted Label'] = [0, 1, 0, 0, 0, 0, 1, 1, 1, 1]
df['Actual Label'] = df['Target']

misclassified_samples = len(df[df['Predicted Label'] != df
['Actual Label']])
eps = 0.1 * misclassified_samples
print("Epsilon:", eps)
```

Epsilon: 0.30000000000000004

```
import numpy as np

learn_weights = 0.5 * np.log((1 - eps) / eps)
print("Learner Weights:", learn_weights)
```

Learner Weights: 0.4236489301936017

```

# Misclassified Samples
df.loc[df['Predicted Label'] != df['Actual Label'], 'Weight'] = 0.1 * np.exp(learn_weights)
# Correctly classified Samples
df.loc[df['Predicted Label'] == df['Actual Label'], 'Weight'] = 0.1 * np.exp(-learn_weights)
print(df[['Actual Label', 'Predicted Label', 'Weight']])
# Compute total weights
total_weight = df['Weight'].sum()
print("Total Weight:", total_weight)

```

	Actual Label	Predicted Label	Weight
0	0	0	0.065465
1	0	1	0.152753
2	0	0	0.065465
3	0	0	0.065465
4	0	0	0.065465
5	1	0	0.152753
6	1	1	0.065465
7	1	1	0.065465
8	1	1	0.065465
9	0	1	0.152753

Total Weight: 0.9165151389911681

```
# Normalize the weight
df.loc[df['Predicted Label'] != df['Actual Label'], 'Weight'] = df['Weight'] / total_weight
df.loc[df['Predicted Label'] == df['Actual Label'], 'Weight'] = df['Weight'] / total_weight
print(df[['Actual Label', 'Predicted Label', 'Weight']])
total_weight = df['Weight'].sum()
print("Total Weight:", total_weight)
```

	Actual Label	Predicted Label	Weight
0	0	0	0.071429
1	0	1	0.166667
2	0	0	0.071429
3	0	0	0.071429
4	0	0	0.071429
5	1	0	0.166667
6	1	1	0.071429
7	1	1	0.071429
8	1	1	0.071429
9	0	1	0.166667

Total Weight: 0.9999999999999999

AdaBoost doesn't say: "Let's fix the model." → It says: "Let's change what the model pays attention to."

So when the next learner arrives. It doesn't see 10 equal samples anymore. It sees:

- 3 samples screaming for attention
- 7 samples quietly fading into the background

And this process repeats. Again. Again. Again. Each iteration:

- Re-evaluates mistakes
- Re-weights importance
- Refocuses attention

By the end, the model doesn't just improve. It becomes **selective**. It learns:

- Which patterns are easy
- Which patterns are deceptive

- Which data points define the boundary

And that's the true power of AdaBoost. Not just combining weak learners. But **forcing each learner to look where others failed**.

That's not just learning. That's **refined observation**. And that is exactly how I would do it for most problems.

Now this is a really sharp question. Because at first glance, it almost feels like AdaBoost is doing something risky. We reduce the weight of correctly classified samples. So naturally, we might worry:

"Won't the next learner ignore them... and start making mistakes on them?"

And the answer is: Yes!! That can happen. And surprisingly, that's completely okay.

What Actually Happens? Let's think about what we're doing. In AdaBoost, we don't remove correctly classified samples. We don't throw them away. We simply say:

"You are less important for now."

So the next learner will pay more attention to the difficult samples, but it still sees the entire dataset.

The key idea is: Because we reduced their weight, a previously correct sample might now be misclassified. And instead of being a problem. This is exactly how AdaBoost **self-corrects**.

The moment that happens:

- That sample is now misclassified
- Its weight immediately increases
- The next learner is forced to focus on it again

We are not ignoring data. We are **shifting attention over time**. Think of it like this:

- Easy samples → temporarily less focus
- Hard samples → temporarily more focus

But nothing is permanent. Every sample is always part of the system just with changing importance.

Imagine I am teaching a class. After a test:

- Some students did well → I spend less time with them
- Some students struggled → I focus more on them

Now in the next test: A “good” student suddenly makes a mistake. Do I ignore them? Of course not. I immediately react: “You need attention now”. And in the next round, they become a priority.

AdaBoost is not trying to be perfect at every step. It allows small mistakes to happen so that it can **detect where attention is needed next**.

Over many iterations:

- Every difficult sample gets highlighted
- Every overlooked mistake gets corrected
- Every sample gets attention **when it needs it**

So instead of treating all data equally all the time. We do something smarter. We treat data **unequally, but fairly over time**.

Apparently reducing the weight of correctly classified samples can lead to future mistakes. But that's not a flaw. That's the mechanism. Because AdaBoost doesn't aim for perfection in one step. It builds a system that constantly asks: *“Who needs attention now?”*

And by answering that question, again and again. We end up with a model that doesn't just learn... But **continuously rebalances its understanding of the data**.

- **Step 5 — Training the Next Weak Learner**

Now we move forward, but notice something very important.

We are not changing the dataset structure: Same features. Same target labels. Same data points. Nothing new is introduced.

The only thing that changes is **how much we care about each sample**.

Training the Second Learner

I take the updated weights from the previous step and train a new weak learner. And this time, something interesting happens.

True Label	Predicted Label	Evaluation
0	0	
0	0	
0	0	
0	0	
0	1	Incorrect
1	1	
1	1	
1	1	
1	0	Incorrect
0	0	

Some of the previously misclassified samples are now **correctly classified**. That tells me something very important: The weight adjustment worked. The learner actually paid attention to the difficult cases.

But not everything is perfect. There are still mistakes. And even more interesting: ***Some samples that were previously correct are now misclassified.***

What Is Happening Here? At first, this might feel wrong.

| ***“But wait... didn’t we already get those right?”***

Yes. But remember what we did: We reduced their weights. We told the model: “These are less important for now”. So the new learner focused elsewhere, and as a result, it might “forget” some of the easier cases.

And here’s the key idea: This is not a bug. This is the **design**. Because AdaBoost is not trying to be perfect at every step. It is constantly asking: “Where should I focus *now*?”

The Cycle Continues

So once again, we repeat the same logic:

- Misclassified samples → increase their weights
- Correctly classified samples → decrease their weights

And then we train the next learner on this **new perspective of the data**.

Evaluate the Second Learner - This time, we only have **2 misclassified samples**. So the weighted error becomes:

$$\epsilon = 0.1 \times 2 = 0.2$$

→ Even better than before.

```
# The only difference is the predicted label
df['Predicted Label2'] = [0, 0, 0, 0, 1, 1, 1, 1, 0, 0]

misclassified_samples2 = len(df[df['Predicted Label2'] != df
['Actual Label']])
eps2 = 0.1 * misclassified_samples2
print("Epsilon 2:", eps2)
```

Epsilon 2: 0.2

Now we compute the learner's influence:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon}{\epsilon} \right)$$

Substitute $\epsilon = 0.2 \rightarrow \alpha_t \approx 0.6931$

```
import numpy as np

learn_weights2 = 0.5 * np.log((1 - eps2) / eps2)
print("Learner Weights 2:", learn_weights2)
```

Learner Weights 2: 0.6931471805599453

Now pause for a second. Compare this with the first learner:

- First $\alpha_t \approx 0.4236$
- Second $\alpha_t \approx 0.6931$

→ This learner is **stronger**. Which means it made fewer mistakes, and it will have **more influence** in the final decision

- **Update the Sample Weights Again:** Now I repeat the same process, but with new values.

(A) Misclassified Samples: These are now the most critical points.

$$w_{new} = w_{old} \times e^{(+\alpha_t)}$$

For the two misclassified samples → Their weights jump to ≈ 0.1428579

(B) Correctly Classified Samples

Now we have two groups:

- Previously difficult samples (higher weight before)
- Consistently correct samples (lower weight before)

Both are reduced, but not equally impactful.

After applying:

$$w_{new} = w_{old} \times e^{(-\alpha_t)}$$

We get:

- Previously difficult → ≈ 0.0833329
- Always correct → ≈ 0.0357145


```

# Misclassified Samples
df.loc[df['Predicted Label2'] != df['Actual Label'], 'Weight'] = df['Weight'] * np.exp(learn_weights2)
# Classified samples
df.loc[df['Predicted Label2'] == df['Actual Label'], 'Weight'] = df['Weight'] * np.exp(-learn_weights2)

print(df[['Actual Label', 'Predicted Label2', 'Weight']])

total_weight = df['Weight'].sum()
print("Total Weight:", total_weight)

```

	Actual Label	Predicted Label2	Weight
0	0	0	0.035714
1	0	0	0.083333
2	0	0	0.035714
3	0	0	0.035714
4	0	1	0.142857
5	1	1	0.083333
6	1	1	0.035714
7	1	1	0.035714
8	1	0	0.142857
9	0	0	0.083333

Total Weight: 0.7142857142857144

- **Normalize Again** - Total weight becomes: $W \approx 0.714287$. So we normalize:
 - Misclassified $\rightarrow \approx 0.20$
 - Previously misclassified (now correct) $\rightarrow \approx 0.1167$
 - Always correct $\rightarrow \approx 0.05$

```
# Normalize the weight
df.loc[df['Predicted Label2'] != df['Actual Label'], 'Weight'] = df['Weight'] / total_weight
df.loc[df['Predicted Label2'] == df['Actual Label'], 'Weight'] = df['Weight'] / total_weight
print("\nNormalized:\n", df[['Actual Label', 'Predicted Label2', 'Weight']])
total_weight = df['Weight'].sum()
print("Total Weight:", total_weight)
```

Normalized:

	Actual Label	Predicted Label2	Weight
0	0	0	0.050000
1	0	0	0.116667
2	0	0	0.050000
3	0	0	0.050000
4	0	1	0.200000
5	1	1	0.116667
6	1	1	0.050000
7	1	1	0.050000
8	1	0	0.200000
9	0	0	0.116667

Total Weight: 0.9999999999999998

Now Look at the Pattern (This Is Important) → We now have **three levels of importance**:

- **Highest weight (~0.20)** → Misclassified *in this round* → These are the hardest cases right now
- **Medium weight (~0.1167)** → Previously misclassified, now corrected → Still important, but less urgent
- **Lowest weight (~0.05)** → Correct across both learners → These are "easy" samples

What Does This Tell Us? AdaBoost is not just separating correct vs incorrect. It is building a **hierarchy of difficulty**. Not all samples are treated equally anymore. We now have:

- Critical cases
- Moderate cases

- Easy cases

At this point, I'm no longer just training models. I'm watching the system **learn how to prioritize**. Each iteration answers: "What is the hardest part of this dataset right now?". And then shifts focus accordingly.

If I think like a teacher:

- Students who fail now → get the most attention
- Students who struggled before → still get some attention
- Students who always succeed → minimal attention

But the moment any student struggles. They move **up the priority list immediately**.

Over multiple iterations, this creates something powerful: The model doesn't just improve accuracy. It develops a **dynamic understanding of difficulty**. And by continuously rebalancing attention. We end up with a system that doesn't just learn patterns— It learns **where learning is needed most**.

And that is how weak learners, step by step, turn into something strong.

3. Adaptive Boosting with Python

Here's how you would build an AdaBoost model in Python using the Scikit-Learn library:

```

# Adaptive Boosting Classifier (or AdaBoost)
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score

# Prepare the data
df = load_iris()
X = df.data
y = df.target
# Training and Testing Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Train the model
model = AdaBoostClassifier(n_estimators=100, learning_rate=1, random_state=42)
model.fit(X_train, y_train)
# Predict and calculate accuracy
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy Score: {accuracy}")

```

Accuracy Score: 0.9333333333333333

Summary

AdaBoost and Gradient Boosting may look similar on the surface...

- Both build models **sequentially**.
- Both learn from previous iterations.
- Both combine weak learners into something strong.

So in terms of the *learning method*, they are the same. But their **philosophy** is completely different.

- AdaBoost says: "Let's change the importance of the data."

- Gradient Boosting says: "Let's directly fix the mistakes."

So while they share the same *framework*. They differ in:

- **Idea** → attention vs correction
- **Execution** → re-weighting samples vs modeling residuals

If I had to compress everything into one line: *AdaBoost and Gradient Boosting learn the same way, but **think in completely different ways**.*

References

<https://www.geeksforgeeks.org/machine-learning/adaboost-in-machine-learning/>

<https://www.mygreatlearning.com/blog/adaboost-algorithm/>

<https://www.kaggle.com/code/prashant111/adaboost-classifier-tutorial>

<https://www.youtube.com/watch?v=LsK-xG1cLYA>

Next Section:

 13. Logistic Regression